

Experiment Plan: Sparse Signal-Field SAEs for Hierarchical Vision Representations

1 Purpose

This document outlines the planned experiment for testing whether intermediate vision-model activations can be decomposed into sparse, reusable, interpretable feature coefficients whose spatial or patchwise support is also sparse in a transformed field.

The central goal is not merely to reconstruct hidden activations with an autoencoder. The goal is to test whether a vision model’s internal activation fields can be rewritten into a sparse transform-domain representation, masked in that transformed space, and still reconstructed well enough to preserve downstream computation.

The intended model family for the first experiments is a small ResNet-style CNN. The same framework is later intended to transfer to larger CNNs and Vision Transformers (ViTs), after the best-performing small-model configuration is selected.

2 Premise and Intuition

2.1 Images are not sparse in pixels, but they are compressible in transformed spaces

Natural images are usually dense in their raw pixel basis. A typical image does not contain only a few nonzero pixels. However, images are often sparse or compressible after transformation into a more appropriate basis. Classical examples include Fourier, DCT, wavelets, curvelets, shearlets, and learned sparse dictionaries.

The guiding analogy is wavelet compression. A natural image may be dense in pixels, but after a wavelet transform much of the useful signal may be concentrated in a relatively small set of coefficients. These coefficients are indexed not only by value but also by scale, orientation, and spatial location.

This project asks whether a similar idea applies to neural activations:

$$\text{dense model activation} \longrightarrow \text{sparse learned coefficient field.}$$

The sparse autoencoder (SAE) plays the role of a learned transform. The mask plays the role of selecting a sparse set of coefficients in the transformed field.

2.2 Basis-wise sparsity and signal-location sparsity

There are two different sparsity claims in the project.

Basis-wise sparsity. The activation may be dense in the model’s native channel space but sparse in a learned feature basis. This is the standard sparse-coding or SAE assumption. The SAE learns a large dictionary of possible features, but each example should use only a sparse subset of those features.

Signal-location sparsity. After the activation is represented in the learned sparse basis, the important signal may occupy only a sparse set of locations in the transformed field. In other words, the model may not need all feature-location coefficients to reconstruct or preserve the computation.

The experiment therefore separates:

what feature is active

from

where in the field that transformed coefficient is active.

2.3 Sparse feature fields

For a hidden activation field at section Q_t , let:

$$\mathbf{H}_t \in \mathbb{R}^{B \times C_t \times H_t \times W_t},$$

where B is batch size, C_t is the channel dimension, and $H_t \times W_t$ is the spatial field.

The SAE encoder maps this activation field into:

$$\mathbf{Z}_t \in \mathbb{R}^{B \times K_t \times H_t \times W_t},$$

where K_t is the number of learned sparse features.

The coefficient $\mathbf{Z}_t[b, k, h, w]$ means:

feature k is active at field location (h, w) for example b .

The same feature dictionary is shared across the field. Thus feature k means the same learned feature type at every location. This is essential. It prevents the model from learning unrelated location-specific features and allows us to interpret $\mathbf{Z}_t[k, :, :]$ as a spatial coefficient map for feature k .

2.4 The wavelet analogy

The analogy is not that SAE features literally correspond to semantic image parts. The analogy is structural:

wavelet coefficient = basis type/scale/location coefficient,

whereas here:

SAE coefficient = learned feature type/location coefficient.

The mask is applied after the SAE transform:

$$\tilde{\mathbf{Z}}_t = \mathbf{M}_t \odot \mathbf{Z}_t.$$

Thus the mask removes entries in the transformed coefficient field, not raw pixels, raw CNN channels, or raw ViT tokens.

2.5 Sparsity tree intuition

The hierarchy hypothesis is that sparse coefficients at an earlier section Q_t should support, predict, or causally generate sparse coefficients at the next section Q_{t+1} .

This is the “sparsity tree” intuition. It is not a hand-coded semantic tree such as object $\rightarrow$ part $\rightarrow$ subpart. Instead, it is a learned coefficient hierarchy:

$$\mathbf{Z}_t \longrightarrow \mathbf{Z}_{t+1}.$$

The parent and child nodes are sparse feature-location coefficients in learned activation space. The desired evidence is that a sparse masked representation at Q_t can reconstruct or predict the sparse representation at Q_{t+1} , and that this remains true when chained through later sections up to Q_5 .

2.6 Important caveat

The mask should not be overinterpreted. A feature’s activation location is not always the same as the image evidence location that caused the feature, especially in ViTs where attention can route information nonlocally. The project should distinguish:

1. **Activation location:** where a feature coefficient is represented.
2. **Evidence location:** which image region causally produced it.
3. **Decision-impact location:** which feature-location interventions affect the final prediction.

The present experiments mainly test reconstruction and sparse transformed-field sufficiency. Causal interpretation requires later intervention tests.

3 Overall Experimental Setup

3.1 Sections $Q_1, \dots, Q_5$

For the initial ResNet CNN experiments, choose 4–5 post-activation hidden sections:

$$Q_1, Q_2, Q_3, Q_4, Q_5.$$

Each Q_t is a post-activation hidden map in the frozen CNN. The final section Q_5 should be the last spatial hidden representation before global pooling or the classifier MLP/head.

At each section, collect:

$$\mathbf{H}_t = \text{post-activation hidden map at } Q_t.$$

Each section has its own spatial resolution, channel width, and SAE:

$$E_t, D_t.$$

The SAE encodes and decodes:

$$\mathbf{H}_t \xrightarrow{E_t} \mathbf{Z}_t \xrightarrow{\text{mask}} \tilde{\mathbf{Z}}_t \xrightarrow{D_t} \hat{\mathbf{H}}_t.$$

The reconstruction objective is:

$$\mathcal{L}_{\text{recon},t} = \left\| \mathbf{H}_t - \hat{\mathbf{H}}_t \right\|_2^2,$$

possibly normalized by the activation energy.

A sparsity or masking objective is applied so that reconstruction must use only a sparse set of transformed coefficients.

3.2 What is being tested

The project tests three nested claims:

1. **Layerwise sparse reconstruction:** Can each section Q_t 's hidden state be reconstructed from masked sparse SAE coefficients?
2. **Cross-section sparse prediction:** Can sparse masked coefficients at Q_t predict sparse coefficients at Q_{t+1} ?
3. **Chained sparse hierarchy:** Can this be chained from Q_1 to Q_5 , so that the final hidden representation or task behavior remains recoverable from the sparse hierarchy?

4 SAE Type A: High-Accuracy Expressive Patch/Convolutional Field SAE

4.1 Motivation

The first SAE family is the main high-accuracy field-preserving SAE. It is inspired by convolutional sparse coding and wavelet-like local transforms. It preserves spatial geometry and transforms the activation map into K_t sparse coefficient maps.

The goal is not merely to linearly expand channels. The goal is to learn a sufficiently expressive transform space so that the hidden map can be reconstructed with high fidelity even after sparse coefficient masking. Therefore the main convolutional SAE should be stronger than a minimal 1×1 or $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ model.

The recommended design is a moderately deep local residual convolutional SAE:

activation map $\rightarrow$ local residual encoder $\rightarrow$ sparse coefficient maps $\rightarrow$ mask $\rightarrow$ local residual decoder $\rightarrow$ reconstruction

This is the best first serious architecture if the priority is high reconstruction accuracy while still preserving the transformed-field masking interpretation.

4.2 Default high-accuracy architecture

For section Q_t , the input is:

$$\mathbf{H}_t \in \mathbb{R}^{B \times C_t \times H_t \times W_t}.$$

The encoder should output:

$$\mathbf{Z}_t \in \mathbb{R}^{B \times K_t \times H_t \times W_t}.$$

The recommended default encoder is:

$$\begin{aligned} \mathbf{H}_t &\rightarrow \text{Norm}(C_t) \\ &\rightarrow 1 \times 1 \text{ Conv}(C_t \rightarrow d_t) \\ &\rightarrow \text{GELU} \\ &\rightarrow \text{LocalResBlock}_1 \\ &\rightarrow \text{LocalResBlock}_2 \\ &\rightarrow \text{LocalResBlock}_3 \\ &\rightarrow 1 \times 1 \text{ Conv}(d_t \rightarrow K_t) \\ &\rightarrow \text{sparsity or pre-mask coefficient field.} \end{aligned}$$

The recommended default decoder is:

$$\begin{aligned}
\tilde{\mathbf{Z}}_t &\rightarrow 1 \times 1 \text{ Conv}(K_t \rightarrow d_t) \\
&\rightarrow \text{GELU} \\
&\rightarrow \text{LocalResBlock}_1 \\
&\rightarrow \text{LocalResBlock}_2 \\
&\rightarrow \text{LocalResBlock}_3 \\
&\rightarrow 1 \times 1 \text{ Conv}(d_t \rightarrow C_t) \\
&\rightarrow \hat{\mathbf{H}}_t.
\end{aligned}$$

Here d_t is the hidden width of the SAE. A good default is:

$$d_t = 2C_t$$

for small sections, and:

$$d_t = 4C_t$$

if unmasked reconstruction is poor and memory allows.

The latent width should be overcomplete:

$$K_t \in \{8C_t, 16C_t\}$$

for the first serious masked-reconstruction experiments. If memory is tight, start with $K_t = 4C_t$, but the high-accuracy target should use a larger K_t because sparse reconstruction usually benefits from an overcomplete coefficient dictionary.

4.3 Local residual block

A recommended LocalResBlock is a ConvNeXt-style spatial block:

$$\begin{aligned}
y = x + \text{DropPath} \Big(&1 \times 1 \text{ Conv}(4d_t \rightarrow d_t) \circ \text{GELU} \circ 1 \times 1 \text{ Conv}(d_t \rightarrow 4d_t) \\
&\circ \text{Norm} \circ \text{DWConv}_{5 \times 5}(d_t \rightarrow d_t) \Big)(x).
\end{aligned}$$

In words, each block is:

depthwise 5×5 conv $\rightarrow$ normalization $\rightarrow 1 \times 1$ expansion $\rightarrow$ GELU $\rightarrow 1 \times 1$ projection $\rightarrow$ residual add.

A 5×5 depthwise convolution is recommended over a single 3×3 convolution for the high-accuracy model because it gives the SAE a larger local receptive field while still preserving spatial resolution. If memory or speed is an issue, use 3×3 . If reconstruction is still weak, try dilation or a 7×7 depthwise convolution in later sections.

All spatial convolutions should use stride 1 and appropriate padding, so:

$$H_t \times W_t \text{ stays } H_t \times W_t.$$

4.4 Why this is stronger than the minimal conv SAE

The minimal architecture:

$$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$$

may be useful as a baseline, but it is likely too weak as the main model. It gives only one shallow local mixing step before the sparse bottleneck.

The recommended model uses multiple nonlinear local residual blocks before the sparse coefficient field. This gives the encoder enough capacity to learn a real transform-domain representation of the hidden map, closer in spirit to wavelets or other learned analysis transforms.

The decoder is also moderately expressive so it can synthesize the hidden map from sparse transformed coefficients. However, the decoder is still constrained to be local and field-preserving. It should not be an arbitrary global MLP that can freely inpaint or memorize the whole activation map.

4.5 No shortcut around the sparse bottleneck

Do not use input-to-output skip connections around the sparse coefficient field.

Bad:

$$\mathbf{H}_t \rightarrow E_t \rightarrow \mathbf{Z}_t \rightarrow D_t \rightarrow \hat{\mathbf{H}}_t, \quad \mathbf{H}_t \rightarrow \hat{\mathbf{H}}_t \text{ skip connection.}$$

Such a bypass would allow the autoencoder to reconstruct without relying on the sparse transformed coefficients.

Internal residual connections inside the encoder and decoder blocks are allowed and recommended. They improve optimization while still forcing all information to pass through $\mathbf{Z}_t$.

4.6 Mask placement

The mask is applied only after the encoder has produced the transformed coefficient field:

$$\mathbf{Z}_t = E_t(\mathbf{H}_t), \quad \tilde{\mathbf{Z}}_t = \mathbf{M}_t \odot \mathbf{Z}_t, \quad \hat{\mathbf{H}}_t = D_t(\tilde{\mathbf{Z}}_t).$$

The main mask has shape:

$$\mathbf{M}_t \in \{0, 1\}^{B \times K_t \times H_t \times W_t}.$$

Thus it masks transformed coefficients, not raw pixels or raw channels.

4.7 Training schedule for high reconstruction accuracy

To maximize the chance of high-accuracy reconstruction, do not train from a harsh final TopK mask immediately. Use a staged curriculum.

Stage 0: normalization. Normalize the hidden maps per section before training. At minimum, subtract the dataset mean and divide by the dataset standard deviation per channel. Store the normalization statistics for evaluation.

Stage 1: unmasked or lightly sparse SAE warm-up. Train:

$$\mathbf{H}_t \rightarrow E_t(\mathbf{H}_t) = \mathbf{Z}_t \rightarrow D_t(\mathbf{Z}_t) = \hat{\mathbf{H}}_t$$

with no hard mask or with very light sparsity. This tests whether the architecture has enough capacity. If this stage cannot reconstruct well, the model is underpowered and masking should not be blamed.

Stage 2: mild masking. Introduce a generous TopK mask, retaining many coefficients. This teaches the decoder to reconstruct from masked coefficients without making the problem impossible.

Stage 3: anneal to the target mask. Gradually reduce the number of retained coefficients until reaching the target global or receptive-field TopK budget.

Stage 4: optional fine-tuning. Fine-tune with the final mask budget. If downstream preservation matters, add a small logit-matching or task-preservation term after the reconstruction loss is stable.

4.8 Recommended reconstruction loss

Use a reconstruction loss that is robust to scale differences across sections:

$$\mathcal{L}_{\text{recon}} = \frac{\|\mathbf{H}_t - \hat{\mathbf{H}}_t\|_2^2}{\|\mathbf{H}_t\|_2^2 + \epsilon} + \alpha \left(1 - \cos(\mathbf{H}_t, \hat{\mathbf{H}}_t)\right).$$

A good starting point is $\alpha = 0.1$. If downstream behavior is important during SAE training, add:

$$\beta \text{KL}(f_{\text{orig}}(x) \| f_{\text{recon}}(x)),$$

where f_{recon} is the model after replacing the hidden state with $\hat{\mathbf{H}}_t$. This should be added only after the SAE is already reconstructing reasonably well.

4.9 How to decide whether to add more layers

The first diagnostic should be unmasked reconstruction.

- If unmasked reconstruction is poor, the SAE is underpowered. Increase K_t , then d_t , then the number of local residual blocks.
- If unmasked reconstruction is strong but masked reconstruction is poor, the issue is the sparsity/masking budget, not necessarily the architecture. Increase the number of retained coefficients, use a curriculum, or switch to a learned unmasking penalty.
- If reconstruction is strong but features are duplicated, add mild feature-diversity diagnostics or penalties rather than simply increasing capacity.

A practical architecture sweep is:

Baseline:	1×1 SAE
Small conv SAE:	$1 \times 1 \rightarrow 1$ local block $\rightarrow 1 \times 1$
Main high-accuracy SAE:	$1 \times 1 \rightarrow 3$ local residual blocks $\rightarrow 1 \times 1$
Stronger SAE:	$1 \times 1 \rightarrow 4\text{--}6$ local residual blocks $\rightarrow 1 \times 1$.

The main recommended starting point is the three-block version.

4.10 Section-specific recommendations

For early sections Q_1, Q_2 , the field resolution is larger and the features are more local. Use:

$$3 \text{ local residual blocks,} \quad K_t = 8C_t.$$

For later sections Q_4, Q_5 , the field resolution is smaller and the features are more abstract. Use:

$$3\text{--}4 \text{ local residual blocks,} \quad K_t = 8C_t\text{--}16C_t.$$

For the final spatial section Q_5 , high accuracy is especially important because errors directly affect the classifier head. If memory allows, use the stronger setting:

$$d_t = 4C_t, \quad K_t = 16C_t, \quad 4 \text{ local residual blocks.}$$

4.11 Interpretation

The latent coefficient $\mathbf{Z}_t[k, h, w]$ is a learned local transform response centered at spatial location (h, w) . Because weights are shared convolutionally, feature k is the same feature type everywhere in the field.

This is the most wavelet-like SAE family in the experiment. It allows each coefficient to depend on a local neighborhood while still preserving a clean feature-location coefficient field.

5 SAE Type B: Shared Vector / Patchwise TopK SAE

5.1 Motivation

The second SAE family is the safer, more standard SAE. It applies a tried-and-true vector SAE at every patch/location with shared weights.

This avoids inventing a highly custom SAE. The SAE itself is a standard vector sparse autoencoder. The field structure comes from applying it repeatedly over all locations.

5.2 Conceptual architecture

Flatten the spatial field into $N_t = H_t W_t$ locations:

$$\mathbf{H}_t \in \mathbb{R}^{B \times C_t \times H_t \times W_t} \longrightarrow X_t \in \mathbb{R}^{B \times N_t \times C_t}.$$

Apply the same vector SAE to every location:

$$x_t[p] \in \mathbb{R}^{C_t} \longrightarrow z_t[p] \in \mathbb{R}^{K_t} \longrightarrow \hat{x}_t[p] \in \mathbb{R}^{C_t}.$$

Across all locations, this gives:

$$\mathbf{Z}_t \in \mathbb{R}^{B \times N_t \times K_t}.$$

Equivalently:

$$\mathbf{Z}_t \in \mathbb{R}^{B \times K_t \times N_t}.$$

The row/column interpretation is:

$$\mathbf{Z}_t[k, p] = \text{strength of feature } k \text{ at patch/location } p.$$

Thus this is a feature-by-patch coefficient field.

5.3 Why this still preserves location

The location index p is fixed by the field grid. The learned axis is k , the feature axis. Because the same SAE encoder/decoder weights are shared across all p , feature k has the same meaning at every patch.

This gives the desired separation:

$$\text{feature identity} = k, \quad \text{field location} = p.$$

A feature can appear in multiple patches because the same row k can have nonzero values at many columns p .

5.4 Possible sparsity mechanisms

This SAE may use:

- TopK sparsity,
- BatchTopK sparsity,
- JumpReLU,
- ReLU plus L_1 penalty,
- normalized decoder weights,
- dead-feature resampling.

The initial implementation should prefer TopK or ReLU plus L_1 , depending on stability.

6 Mask Type 1: Global Transformed-Field Masking

6.1 Definition

The global mask keeps only a sparse set of coefficients across the whole transformed field.

For a convolutional field SAE:

$$\mathbf{Z}_t \in \mathbb{R}^{B \times K_t \times H_t \times W_t}.$$

For a patchwise vector SAE:

$$\mathbf{Z}_t \in \mathbb{R}^{B \times K_t \times N_t}.$$

The global mask selects the top K_{mask} entries over all feature-location coefficients:

$$(k, h, w) \quad \text{or} \quad (k, p).$$

This can be expressed as:

$$\tilde{\mathbf{Z}}_t = \mathbf{M}_t \odot \mathbf{Z}_t,$$

where $\mathbf{M}_t$ contains ones only at the retained transformed coefficients.

6.2 Interpretation

This mask tests whether the entire hidden activation field is reconstructible from a sparse subset of transformed coefficients. It is the closest analogue to global transform-domain compression.

6.3 Hyperparameter

The number of unmasked coefficients is a hyperparameter. It can be specified as:

- a fixed number of coefficients per image,
- a percentage of all coefficients,
- a target sparsity ratio,
- or a schedule that anneals from less sparse to more sparse.

7 Mask Type 2: Receptive-Field-Local Masking

7.1 Definition

The receptive-field mask enforces sparsity locally with respect to the next section’s receptive field.

For each target location in Q_{t+1} , compute or approximate the corresponding receptive field in Q_t . Then, within that receptive field, keep only the top K_{RF} transformed coefficients from Q_t .

Thus the mask is not global over the whole image. It is local to the region of Q_t that can influence a particular location at Q_{t+1} .

7.2 Motivation

This follows the “sparsity of sparsity” intuition. Instead of requiring the whole image to have only a few active coefficients, the experiment asks whether each next-layer receptive field can be explained by a sparse subset of lower-layer transformed coefficients.

This is more compatible with textures and repeated patterns. A texture feature may appear many times across the image, but inside each higher-level receptive field only a sparse set of lower-level coefficients may be needed.

7.3 Practical implementation

For each transition $Q_t \rightarrow Q_{t+1}$:

1. Determine the spatial mapping from Q_{t+1} locations to receptive-field windows in Q_t .
2. For each target location u in Q_{t+1} , collect source positions $p \in \text{RF}_t(u)$.
3. Among all coefficients (k, p) inside that receptive field, keep the top K_{RF} .
4. Zero out the rest.

For Q_5 , if it is immediately before global pooling/classification, the receptive field may effectively be large or global. In that case, the local mask can either use the receptive field of the classifier input or fall back to a global mask.

8 Training Protocols: Three Distinct Families

The experiment grid must distinguish three different protocols. These are not interchangeable.

1. **Full independent reconstruction only.** Each section Q_t gets its own independently trained SAE. There is no cross-section chain. This is the baseline that asks whether each hidden map can be reconstructed from a sparse masked transform field.
2. **Independent-SAE chain.** Each section Q_t still gets its own independently trained SAE, but after those SAEs are trained, learn a transition from the masked sparse code at Q_t to the independently learned sparse code at Q_{t+1} . This is the chain where every target code is independently grounded first.
3. **Dependent/incremental chain.** The sparse code at Q_{t+1} is not first learned independently. Instead, it is learned from the sparse code at Q_t while still reconstructing the true hidden state at Q_{t+1} . This is the stronger sparsity-tree protocol.

Because there are three training protocols, two SAE types, and two mask types, the correct initial grid is:

$$3 \times 2 \times 2 = 12$$

experiments, not eight.

9 Protocol A: Full Independent Layerwise Reconstruction

9.1 Goal

This protocol trains each section’s SAE independently and stops there. It does not learn any chain between sections.

The goal is to establish whether each hidden map $\mathbf{H}_t$ can be represented and reconstructed from a sparse masked coefficient field:

$$\mathbf{H}_t \rightarrow E_t(\mathbf{H}_t) = \mathbf{Z}_t \rightarrow \tilde{\mathbf{Z}}_t \rightarrow D_t(\tilde{\mathbf{Z}}_t) = \hat{\mathbf{H}}_t.$$

This is the basic reconstruction feasibility test. If this fails, then the chained protocols are not meaningful yet.

9.2 Training

For each Q_t , train an SAE independently:

$$\mathcal{L}_t = \|\mathbf{H}_t - \hat{\mathbf{H}}_t\|_2^2 + \lambda_{\text{sparse}} \mathcal{L}_{\text{sparse}}(\mathbf{Z}_t, \mathbf{M}_t).$$

If hard TopK masking is used, sparsity may be imposed directly by the mask rather than by an additional penalty. If learned masking is used, add an unmasking penalty.

9.3 Output

At the end of this protocol, we have independently trained SAEs:

$$(E_1, D_1), (E_2, D_2), \dots, (E_5, D_5).$$

Each section has its own sparse transform space. No attempt is made to align or predict across sections.

9.4 Interpretation

This protocol asks:

Can each hidden activation field be reconstructed from a sparse subset of learned transformed coefficients?

It is not a hierarchy test. It is the baseline required before the hierarchy tests.

10 Protocol B: Independent-SAE Chain

10.1 Goal

This is the first actual chain protocol. Each section’s sparse feature basis is still learned independently. After those independent spaces exist, learn transitions between them:

$$\tilde{\mathbf{Z}}_t \rightarrow \hat{\mathbf{Z}}_{t+1}.$$

The target $\mathbf{Z}_{t+1}$ is the independently learned code produced by E_{t+1} . This means every target space is independently grounded in the true hidden map before any transition is learned.

10.2 Stage B.1: Train all layerwise SAEs independently

First run Protocol A and freeze or mostly freeze the SAEs:

$$\mathbf{H}_t \rightarrow E_t \rightarrow \mathbf{Z}_t \rightarrow \tilde{\mathbf{Z}}_t \rightarrow D_t \rightarrow \hat{\mathbf{H}}_t.$$

This gives independent target codes:

$$\mathbf{Z}_{t+1} = E_{t+1}(\mathbf{H}_{t+1}).$$

10.3 Stage B.2: Learn sparse transitions between independent spaces

For each adjacent pair, train a transition model:

$$T_t : \tilde{\mathbf{Z}}_t \rightarrow \hat{\mathbf{Z}}_{t+1}.$$

The primary target is the independently learned next sparse code:

$$\mathbf{Z}_{t+1} = E_{t+1}(\mathbf{H}_{t+1}).$$

The transition loss may include both code prediction and decoded hidden-state reconstruction:

$$\mathcal{L}_t^{\text{chain}} = \|\hat{\mathbf{Z}}_{t+1} - \mathbf{Z}_{t+1}\|_2^2 + \beta \|D_{t+1}(\hat{\mathbf{Z}}_{t+1}) - \mathbf{H}_{t+1}\|_2^2.$$

10.4 Stage B.3: Chain from Q_1 to Q_5

After learning the adjacent transitions, run:

$$\tilde{\mathbf{Z}}_1 \rightarrow \hat{\mathbf{Z}}_2 \rightarrow \hat{\mathbf{Z}}_3 \rightarrow \hat{\mathbf{Z}}_4 \rightarrow \hat{\mathbf{Z}}_5.$$

Then decode the final sparse code:

$$D_5(\hat{\mathbf{Z}}_5)$$

and test whether it reconstructs Q_5 and preserves downstream task performance.

10.5 Interpretation

This protocol asks:

If each section has its own independently learned sparse transform space, can sparse transformed coefficients at one section predict the independently learned sparse transformed coefficients at the next section?

This is a clean hierarchy diagnostic because Q_{t+1} 's sparse code is independently validated before the transition tries to predict it. However, it does not force Q_{t+1} 's basis itself to be learned from Q_t .

11 Protocol C: Dependent / Incremental Sparse-Code Chain

11.1 Goal

This is the stronger sparsity-tree protocol. Instead of first learning Q_{t+1} 's sparse code independently, learn a Q_{t+1} sparse code that is produced from the previous sparse code and that reconstructs the true hidden state at Q_{t+1} .

In other words, Q_{t+1} 's sparse field is constrained to be reachable from Q_t 's sparse field.

11.2 Basic transition

Start with a trained Q_1 SAE. Then learn:

$$\tilde{\mathbf{Z}}_1 \rightarrow P_1(\tilde{\mathbf{Z}}_1) = \hat{\mathbf{Z}}_2 \rightarrow D_2(\hat{\mathbf{Z}}_2) = \hat{\mathbf{H}}_2.$$

The target is not an independently learned $\mathbf{Z}_2$. The target is the true hidden state:

$$\mathbf{H}_2.$$

The loss is:

$$\mathcal{L}_{1 \rightarrow 2} = \|D_2(P_1(\tilde{\mathbf{Z}}_1)) - \mathbf{H}_2\|_2^2 + \lambda_{\text{sparse}} \mathcal{L}_{\text{sparse}}(\hat{\mathbf{Z}}_2).$$

Thus Q_2 's sparse feature space is trained under the constraint that it must be generated from Q_1 's sparse feature space.

11.3 Transition module choices

The predictor P_t can be implemented in several ways:

- a small CNN over the sparse coefficient maps;
- a learned ViT or attention/mixer module over sparse feature maps;
- a frozen original-model path: decode $\tilde{\mathbf{Z}}_t$, run the original CNN blocks from Q_t to Q_{t+1} , then encode or map into the next sparse code;
- a hybrid transition that mixes learned sparse-code prediction with decoded hidden-state reconstruction.

The least custom causal version is:

$$\tilde{\mathbf{Z}}_t \rightarrow D_t(\tilde{\mathbf{Z}}_t) = \hat{\mathbf{H}}_t \rightarrow \text{frozen original block } F_t \rightarrow \hat{\mathbf{H}}_{t+1} \rightarrow E_{t+1} \text{ or } P_t \rightarrow \hat{\mathbf{Z}}_{t+1}.$$

The more flexible learned version is:

$$\tilde{\mathbf{Z}}_t \rightarrow P_t \rightarrow \hat{\mathbf{Z}}_{t+1} \rightarrow D_{t+1} \rightarrow \hat{\mathbf{H}}_{t+1}.$$

11.4 Incremental training schedule

Train incrementally:

$$Q_1 \rightarrow Q_2,$$

then:

$$Q_2 \rightarrow Q_3,$$

then:

$$Q_3 \rightarrow Q_4,$$

and finally:

$$Q_4 \rightarrow Q_5.$$

Do not train all levels at once initially. Incremental training keeps the loss landscape local and makes debugging much easier.

If pairwise training fails, try adjacent-pair joint training:

$$(E_t, D_t, P_t, E_{t+1}, D_{t+1})$$

for one transition at a time. Only after adjacent pairs work should full joint training be considered.

11.5 Interpretation

This protocol asks:

Can the sparse feature space at Q_{t+1} be learned as a sparse successor of the sparse feature space at Q_t , while still reconstructing the true Q_{t+1} hidden state?

This is the strongest hierarchy test. It more directly enforces the sparsity tree than the independent-SAE chain.

12 The Twelve Experiments

The corrected initial grid has three factors:

1. Training protocol:

- Full independent layerwise reconstruction only.
- Independent-SAE chain.
- Dependent/incremental sparse-code chain.

2. SAE type:

- Expressive patch/convolutional field SAE.
- Shared vector / patchwise TopK SAE.

3. Mask type:

- Global transformed-field mask.
- Receptive-field-local mask.

This gives:

$$3 \times 2 \times 2 = 12$$

experiments.

Table 1: Twelve core experiments.

ID	Training Protocol	SAE Type	Mask Type	Main Question
E1	Full independent reconstruction only	Expressive patch/conv field SAE	Global mask	Can each hidden field be reconstructed independently from globally sparse conv-like coefficients?
E2	Full independent reconstruction only	Expressive patch/conv field SAE	Receptive-field mask	Can each hidden field be reconstructed independently when sparsity is imposed locally by next-section RF windows?
E3	Full independent reconstruction only	Shared vector / patch-wise SAE	Global mask	Can the safest standard SAE independently reconstruct each section from globally sparse coefficient fields?
E4	Full independent reconstruction only	Shared vector / patch-wise SAE	Receptive-field mask	Can shared feature-by-patch codes reconstruct each section under RF-local sparsity?
E5	Independent-SAE chain	Expressive patch/conv field SAE	Global mask	After independent training, can masked conv-like sparse codes at Q_t predict independently learned codes at Q_{t+1} ?

ID	Training Protocol	SAE Type	Mask Type	Main Question
E6	Independent-SAE chain	Expressive patch/conv field SAE	Receptive-field mask	Can independently learned conv-like sparse fields form a chain under RF-local sparsity?
E7	Independent-SAE chain	Shared vector / patch-wise SAE	Global mask	Can standard shared SAE codes predict the next independently learned sparse code under global masking?
E8	Independent-SAE chain	Shared vector / patch-wise SAE	Receptive-field mask	Can standard shared feature-by-patch codes form a local sparse chain under RF masks?
E9	Dependent/incremental sparse-code chain	Expressive patch/conv field SAE	Global mask	Can later conv-like sparse fields be learned from earlier sparse fields while reconstructing true hidden states?
E10	Dependent/incremental sparse-code chain	Expressive patch/conv field SAE	Receptive-field mask	Can the strongest local transform model learn the sparsity tree under RF-local constraints?
E11	Dependent/incremental sparse-code chain	Shared vector / patch-wise SAE	Global mask	Can a standard shared SAE support dependent sparse-code learning under global compression?
E12	Dependent/incremental sparse-code chain	Shared vector / patch-wise SAE	Receptive-field mask	Can the safest SAE plus RF-local masking learn an incremental sparse hierarchy?

13 Training Details

13.1 Activation collection

For each image, run the frozen ResNet and save:

$$\mathbf{H}_1, \mathbf{H}_2, \mathbf{H}_3, \mathbf{H}_4, \mathbf{H}_5.$$

Use post-activation maps. For ResNet blocks, the section should be defined consistently: for example, after the residual addition and activation at the end of each major stage.

13.2 Normalization

Each Q_t may have different activation scale. Before training the SAE, normalize activations per section. Possible choices:

- subtract dataset mean and divide by dataset standard deviation per channel;
- use RMS normalization per activation vector;
- store normalization statistics and invert them after decoding if needed.

13.3 Hard TopK masking

The first implementation should use hard TopK unmasking, because it cleanly tests whether reconstruction can be achieved from a fixed sparse support.

For global masking:

TopK over all (k, h, w) .

For receptive-field masking:

TopK over (k, h, w) inside each next-layer receptive field.

The TopK budget should be hyperparameter tuned.

13.4 Fallback: learned sparsity mask

If TopK reconstruction fails even on small models, switch to a learned soft or hard-concrete mask. The objective becomes:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda_{\text{unmask}} \sum_i M_i,$$

where M_i is the learned unmasking value for coefficient i .

This changes the question from:

Can a fixed TopK budget work?

to:

Can the model learn how many transformed coefficients it needs while being penalized for unmasking too many?

This is less clean but may be necessary if hard TopK is too brittle.

14 Evaluation Metrics

14.1 Reconstruction fidelity

For each section:

$$\|\mathbf{H}_t - \hat{\mathbf{H}}_t\|_2^2,$$

relative error:

$$\frac{\|\mathbf{H}_t - \hat{\mathbf{H}}_t\|_2}{\|\mathbf{H}_t\|_2},$$

and activation cosine similarity.

14.2 Downstream preservation

Replace the original hidden state with the reconstructed hidden state and continue the frozen model. Measure:

- top-1/top-5 accuracy;
- logit KL divergence from the original model;
- class probability agreement;
- calibration or confidence degradation.

14.3 Sparse-code quality

Measure:

- number of active coefficients;
- fraction of active coefficient field retained;
- distribution of active features per image;
- distribution of active locations per feature;
- decoder feature cosine similarity to detect duplicates;
- feature activation correlation and co-activation.

14.4 Hierarchy quality

For $Q_t \rightarrow Q_{t+1}$, measure:

- $\hat{\mathbf{Z}}_{t+1}$ prediction error;
- decoded Q_{t+1} reconstruction error;
- chain error from $Q_1 \rightarrow Q_5$;
- effect on final classifier output;
- whether masking a parent coefficient predictably damages child coefficients.

14.5 Interpretability checks

For each learned feature k , inspect:

- top activating images;
- top activating field locations;
- decoded feature direction or activation patch;
- feature maps across examples;
- whether similar features are duplicated or genuinely distinct.

These are interpretability proxies, not causal proof.

15 Model Selection Strategy

Run all twelve experiments on small models first. Select the configuration that best balances:

- reconstruction quality;
- downstream task preservation;
- sparsity level;
- feature distinctness;
- stability across random seeds;
- interpretable feature-location maps;
- successful sparse transitions $Q_t \rightarrow Q_{t+1}$.

The best small-model configuration should then be scaled to larger CNNs and eventually ViTs.

If no hard-TopK version works, switch to learned sparsity masks with an unmasking penalty. If independent SAEs work but dependent sparse-code learning fails, keep the independent SAE setup as the main reconstruction result and treat hierarchy as a separate transition-learning problem.

16 How the CNN Experiment Transfers to ViTs

For a ViT, the patch tokens can be reshaped into a spatial patch grid:

$$T \times D \rightarrow H \times W \times D.$$

Then the same framework applies:

$$H \times W \times D \rightarrow H \times W \times K \rightarrow \text{mask} \rightarrow H \times W \times D.$$

The shared vector SAE corresponds to applying the same vector SAE to every patch token. The expressive field SAE corresponds to applying local mixing over the patch grid before producing sparse coefficient maps.

The class token should be handled separately at first:

- leave it unchanged in the first experiments;
- or give it a separate vector SAE;
- do not force it into the spatial grid unless there is a clear reason.

For ViTs, extra caution is required because activation location and evidence location may differ due to attention. Spatial masks should therefore be validated with causal patch interventions later.

17 Main Research Claim

The most defensible research claim is:

Vision-model activations may be dense in their native channel coordinates but compressible in a learned sparse transform domain. By training shared-feature SAEs at intermediate sections and masking sparse coefficient locations in the transformed field, we can test whether a small subset of learned feature-location coefficients is sufficient to reconstruct hidden states and preserve downstream behavior. Cross-section prediction of sparse codes then tests whether later features can be built from earlier sparse feature fields, providing evidence for a learned sparsity tree.

This claim avoids overclaiming. It does not require every feature to be active in only a few locations globally. It does not claim that the mask is a classical compressed-sensing measurement matrix. It does not claim that activation location is automatically causal evidence location. Instead, it defines a testable empirical hypothesis about sparse transformed-field reconstruction and hierarchical sparse-code prediction.

18 Summary

The experiment has three core components and three distinct protocol families:

1. **Sparse feature basis:** Each section Q_t is encoded by an SAE into K_t shared learned features over a spatial field.
2. **Sparse transformed signal support:** A mask keeps only a sparse subset of transformed feature-location coefficients.
3. **Sparse hierarchy:** Sparse codes at Q_t are used to reconstruct or predict sparse codes at Q_{t+1} , eventually reaching Q_5 .

The twelve experiments cross:

$$\text{training protocol} \times \text{SAE type} \times \text{mask type}.$$

The strongest expected configuration may be the expressive patch/conv field SAE with receptive-field-local masking, but the safest baseline is the shared vector SAE with global or RF-local coefficient masking. The project should begin with the safest small-model tests and only scale the configuration that demonstrates reliable sparse reconstruction and hierarchy.

References

- [1] S. Mallat. A theory for multiresolution signal decomposition: the wavelet representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1989.
- [2] E. Candès, J. Romberg, and T. Tao. Robust uncertainty principles: exact signal reconstruction from highly incomplete frequency information. *IEEE Transactions on Information Theory*, 2006.
- [3] B. Olshausen and D. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 1996.
- [4] E. Simoncelli and B. Olshausen. Natural image statistics and neural representation. *Annual Review of Neuroscience*, 2001.
- [5] J. Portilla and E. Simoncelli. A parametric texture model based on joint statistics of complex wavelet coefficients. *International Journal of Computer Vision*, 2000.
- [6] C. Olah, A. Mordvintsev, and L. Schubert. Feature Visualization. *Distill*, 2017.
- [7] C. Olah et al. The Building Blocks of Interpretability. *Distill*, 2018.
- [8] C. Olah et al. Zoom In: An Introduction to Circuits. *Distill*, 2020.
- [9] N. Elhage et al. Toy Models of Superposition. Anthropic, 2022.
- [10] T. Bricken et al. Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. Anthropic, 2023.
- [11] A. Templeton et al. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. Anthropic, 2024.
- [12] PatchSAE / vision sparse autoencoder work on CLIP ViT patch representations.
- [13] SAE-V / vision-model sparse autoencoder intervention work.
- [14] CaFE / causal feature explanation work warning that ViT activation location may differ from causal evidence location.